

transparent volumes, like a light fog, are more likely to have rays like the middle one. How far the ray has to travel through the volume also determines how likely it is for the ray to make it through.

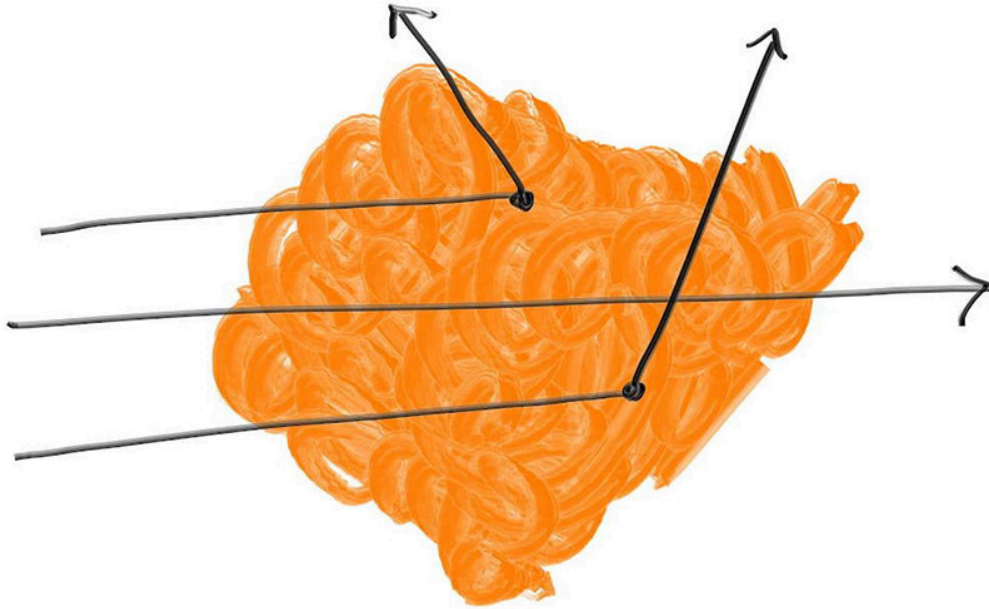


Figure 10: *Ray-volume interaction*

As the ray passes through the volume, it may scatter at any point. The denser the volume, the more likely that is. The probability that the ray scatters in any small distance ΔL is:

$$probability = C \cdot \Delta L$$

where C is proportional to the optical density of the volume. If you go through all the differential equations, for a random number you get a distance where the scattering occurs. If that distance is outside the volume, then there is no “hit”. For a constant volume we just need the density C and the boundary. I’ll use another hittable for the boundary.

The resulting class is:

```

#ifndef CONSTANT_MEDIUM_H
#define CONSTANT_MEDIUM_H

#include "hitable.h"
#include "material.h"
#include "texture.h"

class constant_medium : public hittable {
public:
    constant_medium(shared_ptr<hitable> boundary, double density,
shared_ptr<texture> tex)
        : boundary(boundary), neg_inv_density(-1/density),
          phase_function(make_shared<isotropic>(tex))
    {}

    constant_medium(shared_ptr<hitable> boundary, double density, const color&
albedo)
        : boundary(boundary), neg_inv_density(-1/density),
          phase_function(make_shared<isotropic>(albedo))
    {}

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        hit_record rec1, rec2;

        if (!boundary->hit(r, interval::universe, rec1))
            return false;

        if (!boundary->hit(r, interval(rec1.t+0.0001, infinity), rec2))
            return false;

        if (rec1.t < ray_t.min) rec1.t = ray_t.min;
        if (rec2.t > ray_t.max) rec2.t = ray_t.max;

        if (rec1.t >= rec2.t)
            return false;

        if (rec1.t < 0)
            rec1.t = 0;

        auto ray_length = r.direction().length();
        auto distance_inside_boundary = (rec2.t - rec1.t) * ray_length;
        auto hit_distance = neg_inv_density * std::log(random_double());

        if (hit_distance > distance_inside_boundary)
            return false;

        rec.t = rec1.t + hit_distance / ray_length;
        rec.p = r.at(rec.t);

        rec.normal = vec3(1,0,0); // arbitrary
        rec.front_face = true;    // also arbitrary
        rec.mat = phase_function;

        return true;
    }
}

```

```

aabb bounding_box() const override { return boundary->bounding_box(); }

private:
    shared_ptr<hittable> boundary;
    double neg_inv_density;
    shared_ptr<material> phase_function;
};

#endif

```

Listing 71: [constant_medium.h] *Constant medium class*

The scattering function of isotropic picks a uniform random direction:

```

class diffuse_light : public material {
    ...
};

class isotropic : public material {
public:
    isotropic(const color& albedo) : tex(make_shared<solid_color>(albedo)) {}
    isotropic(shared_ptr<texture> tex) : tex(tex) {}

    bool scatter(const ray& r_in, const hit_record& rec, color& attenuation, ray&
scattered)
    const override {
        scattered = ray(rec.p, random_unit_vector(), r_in.time());
        attenuation = tex->value(rec.u, rec.v, rec.p);
        return true;
    }

private:
    shared_ptr<texture> tex;
};

```

Listing 72: [material.h] *The isotropic class*

The reason we have to be so careful about the logic around the boundary is we need to make sure this works for ray origins inside the volume. In clouds, things bounce around a lot so that is a common case.

In addition, the above code assumes that once a ray exits the constant medium boundary, it will continue forever outside the boundary. Put another way, it assumes that the boundary shape is convex. So this particular implementation will work for boundaries like boxes or spheres, but will not work with toruses or shapes that contain voids. It's possible to write an implementation that handles arbitrary shapes, but we'll leave that as an exercise for the reader.

9.2. Rendering a Cornell Box with Smoke and Fog Boxes

If we replace the two blocks with smoke and fog (dark and light particles), and make the light bigger (and dimmer so it doesn't blow out the scene) for faster convergence:


```

#include "bvh.h"
#include "camera.h"
#include "constant_medium.h"
#include "hitable.h"
#include "hitable_list.h"
#include "material.h"
#include "quad.h"
#include "sphere.h"
#include "texture.h"

...

void cornell_smoke() {
    hitable_list world;

    auto red    = make_shared<lambertian>(color(.65, .05, .05));
    auto white = make_shared<lambertian>(color(.73, .73, .73));
    auto green  = make_shared<lambertian>(color(.12, .45, .15));
    auto light  = make_shared<diffuse_light>(color(7, 7, 7));

    world.add(make_shared<quad>(point3(555,0,0), vec3(0,555,0), vec3(0,0,555),
green));
    world.add(make_shared<quad>(point3(0,0,0), vec3(0,555,0), vec3(0,0,555), red));
    world.add(make_shared<quad>(point3(113,554,127), vec3(330,0,0), vec3(0,0,305),
light));
    world.add(make_shared<quad>(point3(0,555,0), vec3(555,0,0), vec3(0,0,555),
white));
    world.add(make_shared<quad>(point3(0,0,0), vec3(555,0,0), vec3(0,0,555), white));
    world.add(make_shared<quad>(point3(0,0,555), vec3(555,0,0), vec3(0,555,0),
white));

    shared_ptr<hitable> box1 = box(point3(0,0,0), point3(165,330,165), white);
    box1 = make_shared<rotate_y>(box1, 15);
    box1 = make_shared<translate>(box1, vec3(265,0,295));

    shared_ptr<hitable> box2 = box(point3(0,0,0), point3(165,165,165), white);
    box2 = make_shared<rotate_y>(box2, -18);
    box2 = make_shared<translate>(box2, vec3(130,0,65));

    world.add(make_shared<constant_medium>(box1, 0.01, color(0,0,0)));
    world.add(make_shared<constant_medium>(box2, 0.01, color(1,1,1)));

    camera cam;

    cam.aspect_ratio    = 1.0;
    cam.image_width     = 600;
    cam.samples_per_pixel = 200;
    cam.max_depth       = 50;
    cam.background      = color(0,0,0);

    cam.vfov            = 40;
    cam.lookfrom = point3(278, 278, -800);
    cam.lookat  = point3(278, 278, 0);
    cam.vup     = vec3(0,1,0);

    cam.defocus_angle = 0;

```

```
    cam.render(world);  
}  
  
int main() {  
    switch (8) {  
        case 1: bouncing_spheres(); break;  
        case 2: checkered_spheres(); break;  
        case 3: earth(); break;  
        case 4: perlin_spheres(); break;  
        case 5: quads(); break;  
        case 6: simple_light(); break;  
        case 7: cornell_box(); break;  
        case 8: cornell_smoke(); break;  
    }  
}
```

Listing 73: [main.cc] *Cornell box, with smoke*

We get:

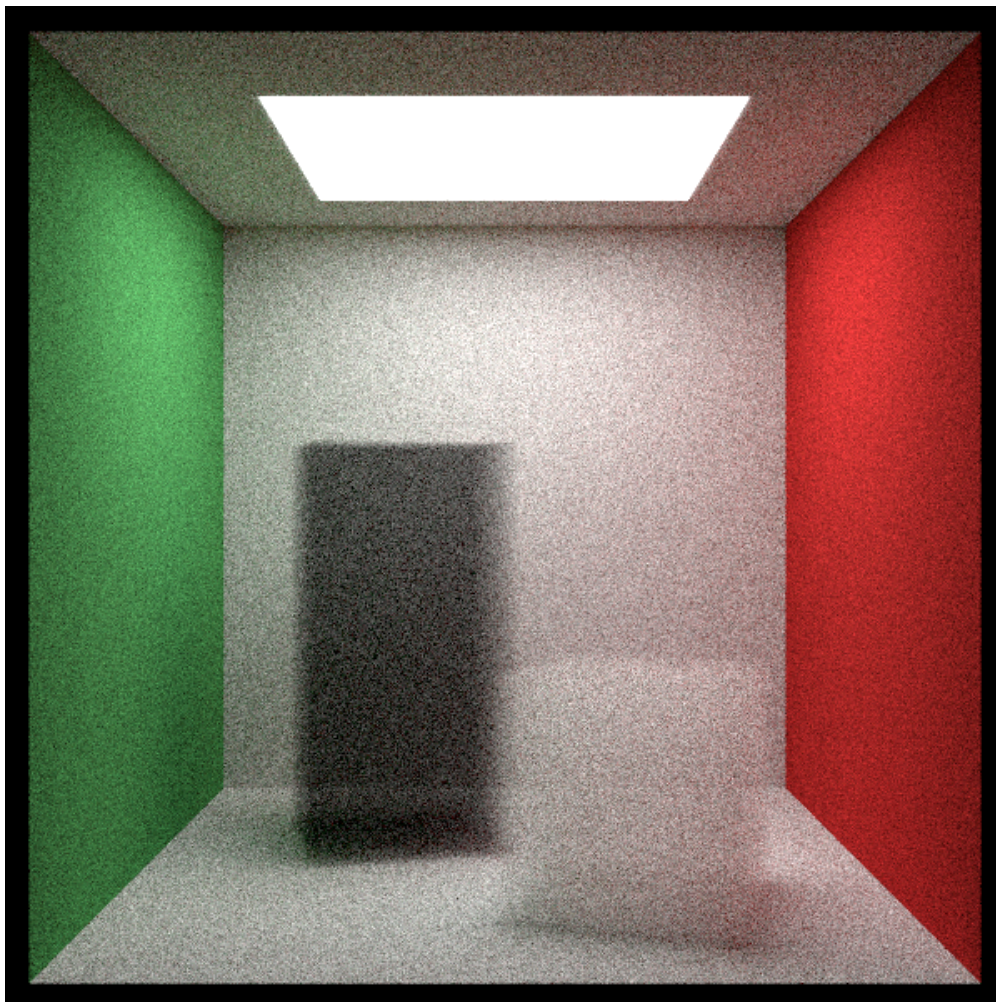


Image 22: *Cornell box with blocks of smoke*

10. A Scene Testing All New Features

Let's put it all together, with a big thin mist covering everything, and a blue subsurface reflection sphere (we didn't implement that explicitly, but a volume inside a dielectric is what a subsurface material is). The biggest limitation left in the renderer is no shadow rays, but that is why we get caustics and subsurface for free. It's a double-edged design decision.

Also note that we'll parameterize this final scene to support a lower quality render for quick testing.

```

void final_scene(int image_width, int samples_per_pixel, int max_depth) {
    hittable_list boxes1;
    auto ground = make_shared<lambertian>(color(0.48, 0.83, 0.53));

    int boxes_per_side = 20;
    for (int i = 0; i < boxes_per_side; i++) {
        for (int j = 0; j < boxes_per_side; j++) {
            auto w = 100.0;
            auto x0 = -1000.0 + i*w;
            auto z0 = -1000.0 + j*w;
            auto y0 = 0.0;
            auto x1 = x0 + w;
            auto y1 = random_double(1,101);
            auto z1 = z0 + w;

            boxes1.add(box(point3(x0,y0,z0), point3(x1,y1,z1), ground));
        }
    }

    hittable_list world;

    world.add(make_shared<bvh_node>(boxes1));

    auto light = make_shared<diffuse_light>(color(7, 7, 7));
    world.add(make_shared<quad>(point3(123,554,147), vec3(300,0,0), vec3(0,0,265),
light));

    auto center1 = point3(400, 400, 200);
    auto center2 = center1 + vec3(30,0,0);
    auto sphere_material = make_shared<lambertian>(color(0.7, 0.3, 0.1));
    world.add(make_shared<sphere>(center1, center2, 50, sphere_material));

    world.add(make_shared<sphere>(point3(260, 150, 45), 50, make_shared<dielectric>
(1.5)));
    world.add(make_shared<sphere>(
        point3(0, 150, 145), 50, make_shared<metal>(color(0.8, 0.8, 0.9), 1.0)
    ));

    auto boundary = make_shared<sphere>(point3(360,150,145), 70,
make_shared<dielectric>(1.5));
    world.add(boundary);
    world.add(make_shared<constant_medium>(boundary, 0.2, color(0.2, 0.4, 0.9)));
    boundary = make_shared<sphere>(point3(0,0,0), 5000, make_shared<dielectric>
(1.5));
    world.add(make_shared<constant_medium>(boundary, .0001, color(1,1,1)));

    auto emat = make_shared<lambertian>(make_shared<image_texture>("earthmap.jpg"));
    world.add(make_shared<sphere>(point3(400,200,400), 100, emat));
    auto pertext = make_shared<noise_texture>(0.2);
    world.add(make_shared<sphere>(point3(220,280,300), 80, make_shared<lambertian>
(pertext)));

    hittable_list boxes2;
    auto white = make_shared<lambertian>(color(.73, .73, .73));
    int ns = 1000;
    for (int j = 0; j < ns; j++) {

```

```

        boxes2.add(make_shared<sphere>(point3::random(0,165), 10, white));
    }

    world.add(make_shared<translate>(
        make_shared<rotate_y>(
            make_shared<bvh_node>(boxes2), 15),
            vec3(-100,270,395)
        )
    );

    camera cam;

    cam.aspect_ratio      = 1.0;
    cam.image_width       = image_width;
    cam.samples_per_pixel = samples_per_pixel;
    cam.max_depth         = max_depth;
    cam.background        = color(0,0,0);

    cam.vfov              = 40;
    cam.lookfrom = point3(478, 278, -600);
    cam.lookat   = point3(278, 278, 0);
    cam.vup      = vec3(0,1,0);

    cam.defocus_angle = 0;

    cam.render(world);
}

int main() {
    switch (9) {
        case 1: bouncing_spheres();      break;
        case 2: checkered_spheres();     break;
        case 3: earth();                  break;
        case 4: perlin_spheres();         break;
        case 5: quads();                  break;
        case 6: simple_light();           break;
        case 7: cornell_box();            break;
        case 8: cornell_smoke();          break;
        case 9: final_scene(800, 10000, 40); break;
        default: final_scene(400, 250, 4); break;
    }
}

```

Listing 74: [main.cc] *Final scene*

Running it with 10,000 rays per pixel (sweet dreams) yields:

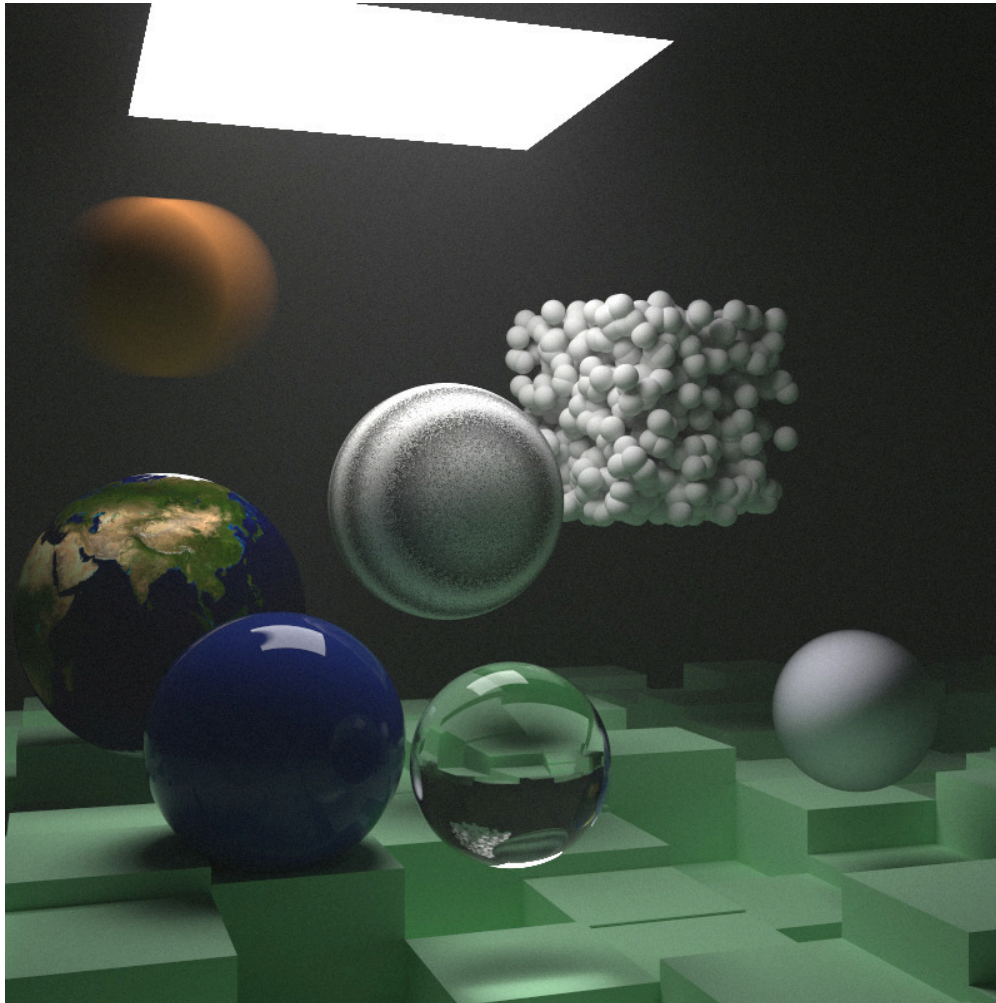


Image 23: *Final scene*

Now go off and make a really cool image of your own! See [our Further Reading wiki page](#) for additional project related resources. Feel free to email questions, comments, and cool images to me at ptrshrl@gmail.com.

11. Acknowledgments

Original Manuscript Help

Dave Hart

Jean Buckley

Web Release

Berna Kabadayı
Lorenzo Mancini

Lori Whippler Hollasch
Ronald Wotzlaw

Corrections and Improvements

Aaryaman Vasishta

Andrew Kensler

Antonio Gamiz

Apoorva Joshi	Gareth Martin	Matthew Heimlich
Aras Prankevičius	Gerrit Wessendorf	Nakata Daisuke
Arman Uguray	Grue Debry	Nate Rupsis
Becker	Gustaf Waldemarson	Niccolò Tiezzi
Ben Kerl	Ingo Wald	Paul Melis
Benjamin Summerton	Jason Stone	Phil Cristensen
Bennett Hardwick	JC-ProgJava	LollipopFt
Benny Tsang	Jean Buckley	Ronald Wotzlaw
Dan Drummond	Jeff Smith	Shaun P. Lee
David Chambers	Joey Cho	Shota Kawajiri
David Hart	John Kilpatrick	Tatsuya Ogawa
Dimitry Ishenko	Kaan Eraslan	Thiago Ize
Dmitry Lomov	Lorenzo Mancini	Thien Tran
Eric Haines	Manas Kale	Vahan Sosoyan
Fabio Sancinetti	Marcus Ottosson	WANG Lei
Filipe Scur	Mark Craig	Yann Herklotz
Frank He	Markus Boos	ZeHao Chen

Special Thanks

Thanks to the team at [Limnu](#) for help on the figures.

These books are entirely written in Morgan McGuire's fantastic and free [Markdeep](#) library. To see what this looks like, view the page source from your browser.

Thanks to [Helen Hu](#) for graciously donating her <https://github.com/RayTracing/> GitHub organization to this project.

12. Citing This Book

Consistent citations make it easier to identify the source, location and versions of this work. If you are citing this book, we ask that you try to use one of the following forms if possible.

12.1. Basic Data

- **Title (series):** “Ray Tracing in One Weekend Series”
- **Title (book):** “Ray Tracing: The Next Week”
- **Author:** Peter Shirley, Trevor David Black, Steve Hollasch
- **Version/Edition:** v4.0.2
- **Date:** 2025-04-25
- **URL (series):** <https://raytracing.github.io>
- **URL (book):** <https://raytracing.github.io/books/raytracingthenextweek.html>

12.2. Snippets

12.2.1 Markdown

```
[_Ray Tracing: The Next Week_]
(https://raytracing.github.io/books/RayTracingTheNextWeek.html)
```

12.2.2 HTML

```
<a href="https://raytracing.github.io/books/RayTracingTheNextWeek.html">
  <cite>Ray Tracing: The Next Week</cite>
</a>
```

12.2.3 LaTeX and BibTeX

```
~\cite{Shirley2025RTW2}

@misc{Shirley2025RTW2,
  title = {Ray Tracing: The Next Week},
  author = {Peter Shirley, Trevor David Black, Steve Hollasch},
  year = {2025},
  month = {April},
  note = {\small
\texttt{https://raytracing.github.io/books/RayTracingTheNextWeek.html}},
  url = {https://raytracing.github.io/books/RayTracingTheNextWeek.html}
}
```

12.2.4 BibLaTeX

```
\usepackage{biblatex}

~\cite{Shirley2025RTW2}

@online{Shirley2025RTW2,
  title = {Ray Tracing: The Next Week},
  author = {Peter Shirley, Trevor David Black, Steve Hollasch},
  year = {2025},
  month = {April},
  url = {https://raytracing.github.io/books/RayTracingTheNextWeek.html}
}
```

12.2.5 IEEE

“Ray Tracing: The Next Week.” raytracing.github.io/books/RayTracingTheNextWeek.html
(accessed MMM. DD, YYYY)



12.2.6 MLA

Ray Tracing: The Next Week. raytracing.github.io/books/RayTracingTheNextWeek.html
Accessed DD MMM. YYYY.



formatted by Markdeep 1.17 